

Practice Assignment on Testing, Linux and Servers

Saturday, 3 January 2026

9:34 AM

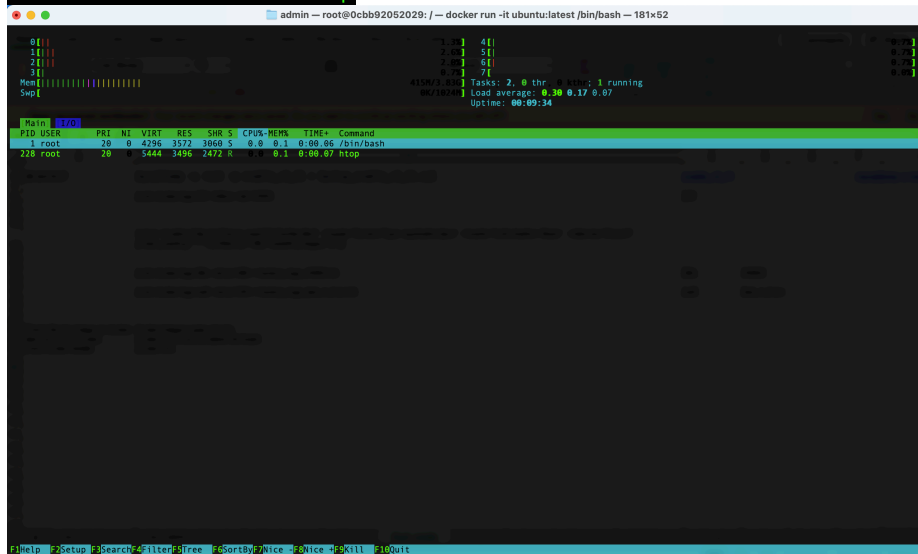
Htop and nmon

Install Monitoring Tools

```
root@cbb92052029:/# apt update
root@cbb92052029:/# apt install htop -y
root@cbb92052029:/# apt install nmon -y
```

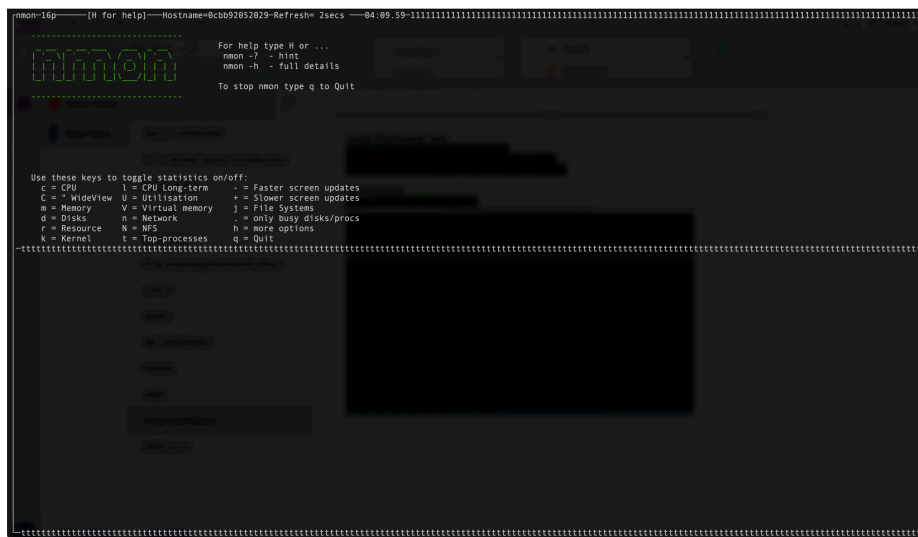
Starting htop

```
root@cbb92052029:/# htop
```



Starting nmon

```
root@cbb92052029:/# nmon
```



Check overall disk usage

```
root@cbb92052029:/# df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay         1007G  3.8G  952G   1% /
tmpfs           64M    0    64M   0% /dev
shm            64M    0    64M   0% /dev/shm
/dev/vda1       1007G  3.8G  952G   1% /etc/hosts
tmpfs           2.0G    0   2.0G   0% /proc/scsi
tmpfs           2.0G    0   2.0G   0% /sys/firmware
```

Check directory-wise usage

```
root@0cbb92052029:/# du -sh /var/*
4.0K   /var/backups
520K   /var/cache
63M    /var/lib
4.0K   /var/local
0      /var/lock
296K   /var/log
4.0K   /var/mail
4.0K   /var/opt
0      /var/run
4.0K   /var/spool
4.0K   /var/tmp
```

Identify Resource-Intensive Processes

```
root@0cbb92052029:/# ps aux --sort=-%mem | head
ps aux --sort=-%cpu | head
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   4296  3572 pts/0    Ss   04:01   0:00 /bin/bash
root       240  50.0  0.0   7628  3444 pts/0    R+   04:17   0:00 ps aux --sort=-%mem
root       241  0.0  0.0   2280  1076 pts/0    S+   04:17   0:00 head
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   4296  3572 pts/0    Ss   04:01   0:00 /bin/bash
root       242  0.0  0.0   7628  3444 pts/0    R+   04:17   0:00 ps aux --sort=-%cpu
root       243  0.0  0.0   2280  1068 pts/0    S+   04:17   0:00 head
```

```
root@0cbb92052029:/# mkdir -p /var/log/system_monitoring
nano /usr/local/bin/system_monitor.sh
```

```
GNU nano 2.2 /usr/local/bin/system_monitor.sh
#!/bin/bash
echo "===== $(date) =====" >> /var/log/system_monitoring/metrics.log
df -h >> /var/log/system_monitoring/metrics.log
free -m >> /var/log/system_monitoring/metrics.log
ps aux --sort=-%cpu | head -5 >> /var/log/system_monitoring/metrics.log
```

```
root@0cbb92052029:/# chmod +x /usr/local/bin/system_monitor.sh
```

```
root@0cbb92052029:/# crontab -e
```

```
GNU nano 2.2 /tmp/crontab.X101YH/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tar.gz /home/
# * * * * /usr/local/bin/system_monitor.sh
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
```

User Management and Access Control

Create User Accounts

```
root@0cbb92052029:/# useradd sarah
root@0cbb92052029:/# useradd mike
```

```
root@0cbb92052029:/# id sarah
uid=1001(sarah) gid=1001(sarah) groups=1001(sarah)
root@0cbb92052029:/# id mike
uid=1002(mike) gid=1002(mike) groups=1002(mike)
root@0cbb92052029:/#
```

Set passwords

```
root@0cbb92052029:/# passwd sarah
New password:
Retype new password:
passwd: password updated successfully
root@0cbb92052029:/#
```

```
root@0cbb92052029:/# passwd mike
New password:
```

```
Retype new password:
passwd: password updated successfully
root@cbb92052029:/#
```

Create Isolated Working Directories

```
root@cbb92052029:/# mkdir -p /home/sarah/workspace
mkdir -p /home/mike/workspace
root@cbb92052029:/#
```

Assign ownership

```
root@cbb92052029:/# chown -R sarah:sarah /home/sarah
chown -R mike:mike /home/mike
root@cbb92052029:/#
```

Set permissions:

```
root@cbb92052029:/# chmod 700 /home/sarah/workspace
chmod 700 /home/mike/workspace
```

```
root@cbb92052029:/# ls -ld /home/*/workspace
drwx----- 2 mike mike 4096 Jan 3 10:00 /home/mike/workspace
drwx----- 2 sarah sarah 4096 Jan 3 10:00 /home/sarah/workspace
```

Enforce Password Expiration Policy

Set password expiration to 30 days

```
root@cbb92052029:/# chage -M 30 sarah
chage -M 30 mike
root@cbb92052029:/#
```

Verify

```
root@cbb92052029:/# chage -l sarah
chage -l mike
Last password change           : Jan 03, 2026
Password expires                : Feb 02, 2026
Password inactive              : never
Account expires                : never
Minimum number of days between password change : 0
Maximum number of days between password change : 30
Number of days of warning before password expires : 7
Last password change           : Jan 03, 2026
Password expires                : Feb 02, 2026
Password inactive              : never
Account expires                : never
Minimum number of days between password change : 0
Maximum number of days between password change : 30
Number of days of warning before password expires : 7
```

Enforce Password Complexity

```
root@cbb92052029:/# nano /etc/security/pwquality.conf
```

```
GNU nano 7.2 /etc/security/pwquality.conf *
minlen = 8
dcredit = -1
ucrcdit = -1
lcredit = -1
ocredit = -1
```

Backup Configuration for Web Servers

Create Backup Directory

```
root@cbb92052029:/# mkdir /backups
chmod 755 /backups
```

Backup Script for Apache (Sarah)

Create script

```
root@cbb92052029:/# nano /usr/local/bin/apache_backup.sh
```

```
GNU nano 7.2 /usr/local/bin/apache_backup.sh *
#!/bin/bash
DATE=$(date +%F)
BACKUP_FILE="/backups/apache_backup_$DATE.tar.gz"
tar -czf $BACKUP_FILE /etc/httpd /var/www/html
```

```
BACKUP_FILE="/backups/apache_backup_${DATE}.tar.gz"
tar -czf $BACKUP_FILE /etc/httpd /var/www/html
tar -tzf $BACKUP_FILE > /backups/apache_verify_${DATE}.log
```

Script Executable

```
root@0cbb92052029:/# chmod +x /usr/local/bin/apache_backup.sh
```

Backup Script for Nginx (Mike)

```
root@0cbb92052029:/# nano /usr/local/bin/nginx_backup.sh
```

```
GNU nano 7.2 /usr/local/bin/nginx_backup.sh *
#!/bin/bash
DATE=$(date +%F)
BACKUP_FILE="/backups/nginx_backup_${DATE}.tar.gz"
tar -czf $BACKUP_FILE /etc/nginx /usr/share/nginx/html
tar -tzf $BACKUP_FILE > /backups/nginx_verify_${DATE}.log
```

Script Executable

```
root@0cbb92052029:/# chmod +x /usr/local/bin/nginx_backup.sh
```

Schedule Cron Jobs (Every Tuesday at 12:00 AM)

```
root@0cbb92052029:/# crontab -e
```

```
GNU nano 7.2 /tmp/crontab.QvvHAQ/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
0 * * * /usr/local/bin/system_monitor.sh
0 0 * * 2 /usr/local/bin/apache_backup.sh
0 0 * * 2 /usr/local/bin/nginx_backup.sh
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
```

Verify Backups Manually

```
root@0cbb92052029:/# ls -ld /backups
drwxr-xr-x 2 root root 4096 Jan  3 10:06 /backups
```

```
root@0cbb92052029:/# ls -lh /backups/
total 0
```

```
root@0cbb92052029:/# mkdir -p /backups
root@0cbb92052029:/# date +%F
2026-01-03
```